iSH TOOLKIT

The complete local operations guide

Version 2.1.0-local

Secrets, SQLite workspaces, portable backups, protected keys and guided recovery.

Built for your iPhone terminal workflow

16 October 2026
Includes upgrade instructions for the working v2 installation.

Read chapters 4 and 5 before upgrading. Do not initialize your existing v2 vault again.

Contents

01 | Purpose and boundaries 3

02 | What changed in 2.1 4

03 | Understand the storage layout 5

04 | Install over your working v2 setup 6

05 | Verify before using live data 7

06 | First-time vault setup and legacy migration 8

07 | Store and retrieve secrets 9

08 | Password changes and TOTP 10

09 | Workspaces and local databases 11

10 | A complete SQLite exercise 12

11 | Two passwords and one key identity 13

12 | Encrypt files and directories 14

13 | Create and verify backups 15

14 | Export to iOS Files 16

15 | Import a portable backup 17

16 | Restore to a separate destination 18

17 | Guided active-vault recovery 19

18 | Interruption and failure recovery 20

19 | A sustainable backup routine 21

20 | Common errors and what they mean 22

21 | Command reference 23

22 | Format notes and trust model 24

01 | Purpose and boundaries

The toolkit turns an iSH terminal into a small, local workspace for secrets, one-time codes, SQLite data and recoverable file storage. Its main purpose is to make repeatable tasks explicit: store a secret, inspect a database, capture a verified snapshot, export that snapshot and recover it later.

It is useful when you want an offline command interface, want to understand where your data lives, or need reproducible operations without a paid backend. It is not a replacement for every feature of a commercial password manager. There is no browser autofill, device synchronization, account recovery service or central administrator.

The build contains working tools for encrypted vault records, TOTP, file encryption, named SQLite workspaces, verified backups and guided vault recovery. Compute commands remain existing status probes. They do not make Docker or GPUs run inside iSH. This release does not include remote databases, automatic publishing, an AI agent or a revenue system.

The important distinction is between storing information and being able to recover it. A local backup is helpful, but an off-device copy plus the necessary passwords and keys is what protects against losing the whole iSH installation. This handbook therefore treats recovery as a routine exercise, not an emergency afterthought.

Use this guide in order for your first setup. Once comfortable, use the command reference and recovery chapters directly. Commands marked with uppercase placeholders require substitution; never type a placeholder as though it were a real backup ID.

02 | What changed in 2.1

The previous v2 build introduced one-file vault storage, transactional password changes, SQLite workspaces and verified local snapshots. Version 2.1 completes the next set of improvements.

Backup export creates one portable .ishbackup file. Import checks its structure and contents before adding it to the local catalogue. This removes the need to manually keep a manifest and payload directory together.

Guided recovery can activate a vault snapshot. It verifies both the snapshot and the current active vault, retains a recovery copy of the current state, then replaces the active records in one transaction. It replaces rather than merges entries. The restored snapshot's password becomes the active vault password.

Crypt-key protection wraps the existing X25519 private key in password-encrypted PKCS8. It preserves the key's identity. Files addressed to that key remain decryptable, provided you know the new key password.

New encrypted files and backups use a streaming authenticated format. The old 32 MiB limit no longer applies to new files. This implementation caps new plaintext streams at 60 GiB, below the per-message GCM bound. Practical limits on iSH will often be lower because of free space, time and device resources.

Historical source backup files have been removed from this distribution. Older installed folders are not deleted. The iSH competing-lock EPERM fix is included.

Do not confuse compatibility with deployment: the new code has passed build-host tests, but your own 2.1 installation must pass its full self-test. The successful 2.0 device test does not prove every new 2.1 path has run on your phone.

03 | Understand the storage layout

The source folder and the data folders serve different purposes. Installed launchers refer to the source scripts; they do not contain a standalone copy of the whole application. Keep the new source folder in place after installation.

The main locations are:

- ~/ish-toolkit-v2.1: this release's code and documentation.
- ~/.local/bin: command launchers created by the installer.
- ~/.vault/vault-v2.db: active encrypted vault records, metadata and TOTP seeds.
- ~/.crypt/id.key: your crypt private key, raw or password-protected.
- ~/.crypt/id.pub: the corresponding raw public key.
- ~/.ish-toolkit/workspaces: named workspace profiles and managed databases.
- ~/.ish-toolkit/backups: verified backup catalogue entries.
- ~/.ish-toolkit/recovery: retained vaults from guided recovery.
- /mnt/export: the iOS Files folder you mounted for transfers.

The vault encrypts secret values and TOTP seeds. Record names remain visible. Ordinary SQLite databases are not automatically encrypted. An encrypted backup protects the snapshot payload, not the live workspace database.

Private directories and files use restrictive permissions where supported. A local process with your account's access can still interact with your files. Since this installation runs as root inside iSH, do not treat Unix permissions as protection from another process already running as the same user.

Never use an ordinary SQL command to modify vault files. Use the vault commands so metadata, record encryption and locks remain consistent. Never delete lock files or SQLite journals to force an operation through.

04 | Install over your working v2 setup

Your fresh v2 vault is already initialized. For this upgrade, do not initialize it again and do not run legacy migration. The installer updates launchers while leaving stored data in place.

Save the archive into the Files folder mounted at /mnt/export. Inspect the real filename:

```
ls -lh /mnt/export
```

iOS previously appended .tar to downloads. Use the exact observed name. This example assumes the original filename; append .tar if your file has that suffix:

```
cd ~
busybox gzip -dc /mnt/export/ish-toolkit-v2.1.tar.gz |
  busybox tar -xvf -
cd ~/ish-toolkit-v2.1
sh install.sh
export PATH="$HOME/.local/bin:$PATH"
hash -r
ish --version
```

Expected version: ish-toolkit 2.1.0-local. Stop if extraction or installation reports an error. Otherwise, later commands could still invoke the old version and give misleading results.

BusyBox extraction is deliberate: it worked on your iSH installation after the other tar path failed. Lines ending with a pipe continue onto the following line. Copy both lines of the extraction command together.

Keep the old source folder until the upgrade is verified. The installer also retains previous launchers. Reverting launchers is not a data conversion: old crypt clients cannot open protected keys or new CRYPTv2 files. A downgrade therefore needs planning, not just a PATH change.

05 | Verify before using live data

Run the full isolated test first:

```
ish selftest --full
```

The command creates a temporary HOME. It exercises filesystem primitives, SQLite policy, backup export/import and restore. The full mode also checks vault rotation, streaming encrypted backup, vault activation with a retained old copy, and protected-key loading. It uses dummy data instead of your live vault or keys.

Let it finish. Crypto and repeated password derivations can take time on iSH. The final line should say that selected self-tests completed and live user data was not used. Any FAIL means stop live key changes and recovery, preserve the output and investigate.

Then verify your real installation:

```
ish vault verify
ish db doctor
```

Vault verification prompts for your existing master password and authenticates every stored record. A fresh empty vault reports zero secrets and zero TOTP entries; that is expected. The database doctor checks read-only policy, parameter binding, rollback and snapshot integrity with temporary data.

The broader doctor can inspect installation state:

```
ish doctor
```

The build-host suite passed 116 tests, including child-process termination around recovery commit and a 34 MiB streaming round trip. These tests are useful engineering evidence, not an independent security audit or a simulation of every power-loss condition. SQLite durability still depends on the device honoring filesystem operations.

06 | First-time vault setup and legacy migration

This chapter is for a new installation or an old legacy vault. Skip initialization and migration when your v2 vault already works, as yours does now.

For a truly empty vault location:

```
ish vault init
ish vault verify
```

Choose a master password when prompted and confirm it. The implementation requires at least eight characters; a longer unique passphrase is preferable. Passwords are not supplied as command-line arguments. There is no password-reset service.

For the legacy three-file format, stop old vault and TOTP clients and preserve a secure copy of the complete old .vault directory. Then:

```
ish vault migrate
ish vault verify
```

Migration authenticates records before publishing the new vault-v2.db. It retains vault.db, salt and verify. Re-running migration on an existing v2 vault verifies it; it does not merge later legacy changes.

Retained legacy files still use their original password. Changing the v2 password does not re-encrypt those files or older backups. Do not resume old clients after migration: their view of the old data can diverge from the new active vault.

The initializer refuses existing vault files. If you want a clean start, first preserve the whole previous directory in a new private location, then initialize explicitly. Never use a delete command just to get past an initialization refusal. Your clean-start workflow already followed this preservation approach.

07 | Store and retrieve secrets

Create an entry with a short descriptive name:

```
ish vault set example-login
```

The command prompts for your master password and the value using hidden input. Names are visible metadata, so avoid putting private information into the name itself. Values can include passwords, tokens or other text you want encrypted.

List names and retrieve a value:

```
ish vault list
ish vault get example-login
```

List shows names without exposing secret values. Get prints the secret to the terminal. Terminal scrollback, screenshots and copied output can retain that plaintext; avoid sharing the output when asking for help.

Setting an existing name updates its value. Deleting an entry requires the master password:

```
ish vault delete example-login
```

Deletion changes the current vault. It does not erase earlier snapshots, retained legacy files or copies outside the toolkit. Back up before substantial edits when you may need the previous state.

To inspect a restored or retained vault without activating it, place the directory option before the subcommand:

```
ish vault --directory ~/recovered-vault verify
ish vault --directory ~/recovered-vault list
ish vault --directory ~/recovered-vault get example-login
```

Use the password belonging to that particular copy. The selected directory is only for that invocation; it does not silently switch the global active vault. This makes it useful for recovery drills and checking preserved data before deciding to activate a snapshot.

08 | Password changes and TOTP

Change the vault master password with:

```
ish vault passwd
ish vault verify
```

You enter the current password, a new password and confirmation. Secrets, TOTP records, salt and verifier change together in one SQLite transaction. If the process is interrupted, the database recovers to the old or committed new transaction. Do not delete a journal. Try the relevant old or new password and verify the vault before further changes.

A password change only affects the active vault. Older snapshots, retained vaults and legacy copies keep their old password. Record which password belongs to each recovery set in a secure place separate from the backup itself.

For a supported account's ordinary Base32 TOTP seed:

```
ish totp add example-account
ish totp list
ish totp example-account
```

The seed is entered at a hidden prompt and stored in the same encrypted vault. The command displays a short-lived code and remaining seconds. With no account argument, ish totp displays codes for stored accounts. Delete an entry with:

```
ish totp delete example-account
```

This client implements the default TOTP parameters used by its pyotp calls; it does not provide a QR scanner or a full provisioning-URI importer. Keep the service's recovery codes separately before changing your authentication setup. A wrong device clock can produce rejected codes, even when the seed is correct.

Do not paste seeds, passwords or current codes into troubleshooting messages. A code's short lifetime does not make its seed safe to share.

09 | Workspaces and local databases

A workspace gives a SQLite database a stable name. It makes commands easier to repeat without relying on a current directory or an implicit selected database.

```
ish workspace create notes
ish workspace list
ish workspace status notes
ish db tables @notes
```

Names start with a lowercase letter and use lowercase letters, digits, underscores or hyphens, with a maximum of 48 characters. A managed workspace includes a database and folders for queries and reports.

To register an existing database instead of copying it:

```
ish workspace create existing --database ~/notes.db
ish db tables @existing
```

Registration points to that file. Moving the original file later can break the profile. It does not create a separate backup.

```
ish workspace path notes
ish workspace open notes
```

These commands print workspace information. Open does not change your shell directory, start a desktop application or establish an implicit active database. Every database operation still names its target explicitly.

Use ordinary SQLite for structured information such as notes, inventories or task records. It is plaintext storage. Passwords and TOTP seeds belong in the vault, not in a workspace table on the assumption that the toolkit encrypts all files.

A workspace backup captures its database, not every file in its queries/reports folders or the workspace profile. Copy important supporting documents separately. A restored database can be registered under a new workspace name after you inspect it.

10 | A complete SQLite exercise

Create a workspace if notes does not already exist:

```
ish workspace create notes
```

Create a small table. Writes deliberately require both flags:

```
ish db query @notes \
  --sql 'CREATE TABLE notes(id INTEGER PRIMARY KEY, body TEXT)' \
  --write --allow-destructive
```

Insert a non-sensitive demonstration value and read it back:

```
ish db query @notes \
  --sql "INSERT INTO notes(body) VALUES ('first note')" \
  --write --allow-destructive
ish db query @notes --sql 'SELECT id, body FROM notes'
ish db schema @notes notes
```

Expected output includes the inserted row as JSON. Running CREATE TABLE again will fail because the table already exists; that is not an installation fault.

For values supplied by another program, use bound parameters rather than building SQL strings. Write a private JSON parameter file and use a question-mark placeholder:

```
umask 077
printf '["another note"]\n' > ~/note-params.json
ish db query @notes \
  --sql 'INSERT INTO notes(body) VALUES (?)' \
  --params-file ~/note-params.json \
  --write --allow-destructive
```

Parameter files themselves are plaintext. Delete or protect them according to their contents. The CLI also accepts SQL with --file and supports stdin with a dash; SQL and parameters cannot both consume stdin in one command.

Read-only is enforced by connection settings and SQLite authorization. Both write flags are required even for a simple insert. The command runs one statement at a time; this is not a general multi-statement migration runner.

11 | Two passwords and one key identity

The vault master password and crypt-key password protect different things.

The vault password derives the key used to encrypt secret records inside vault-v2.db. The crypt private key is a separate X25519 identity used for encrypted files and the outer layer of encrypted backups. A protected private key adds its own password to unlock that identity.

An encrypted vault backup therefore needs the matching crypt private key, its password if protected, and the vault password used when the snapshot was made. Knowing only the current vault password is insufficient if the backup is older or the crypt key is missing.

Check or create a crypt key:

```
ish crypt key-status
ish crypt keygen
```

Key-status requires an existing id.key. Keygen preserves an existing key rather than replacing it. New keygen output starts as a legacy raw key; protection is an explicit next step:

```
ish crypt protect-key
ish crypt key-status
```

Choose and confirm a new crypt-key password of at least 12 characters. If the key was already protected, you also enter its current password. Re-running protect-key changes the wrapping password without changing the public/private key identity.

Back up the resulting protected id.key separately. The public id.pub is useful for recipients but cannot decrypt your files. Old raw copies of the private key remain able to decrypt ciphertext addressed to this identity. Password protection does not revoke those copies or securely erase historical storage blocks.

12 | Encrypt files and directories

New file encryption streams through bounded buffers. It no longer loads an entire large plaintext file into Python memory.

```
ish crypt encrypt ~/notes.txt ~/notes.txt.crypt
ish crypt decrypt ~/notes.txt.crypt ~/notes-restored.txt
```

Both commands refuse an existing output. Choose a new destination to avoid overwriting the original. With a protected key, enter the crypt-key password at the prompt. This is not the vault master-password prompt.

For a directory:

```
ish crypt encrypt-dir ~/documents
```

The command first creates a temporary tar archive and then encrypts it. This requires additional disk space; streaming encryption does not make directory packaging space-free. The output name is the directory path with .tar.crypt appended.

To encrypt for another trusted recipient, use their raw public-key file:

```
ish crypt encrypt -r ~/recipient.pub ~/notes.txt
```

Check that public key through a trusted channel before using it. The matching private key is required for decryption. The pubkey command prints your public key as hexadecimal; the recipient-file argument expects raw bytes such as id.pub, not that printed text.

The new format authenticates the complete header and ciphertext. Plaintext remains in private staging until final tag verification succeeds. Corrupted, truncated or wrong-key input is refused without publishing the requested output. A hard process kill can leave staging files; private staging is not secure erasure.

Old CRYPTv1 backups up to 32 MiB remain readable. Old larger file-encryption outputs need a separate legacy conversion plan before protecting their only raw key. Old clients cannot read CRYPTv2.

13 | Create and verify backups

Use the backup catalogue for snapshots you want to list, verify, export or restore later.

```
ish backup create @notes
ish backup list
```

Create returns an ID containing a UTC timestamp and random suffix. Copy the actual ID. BACKUP_ID in this guide is a placeholder:

```
ish backup verify BACKUP_ID
```

Creating a database snapshot uses SQLite's backup API, including committed WAL contents. The snapshot is checked before it becomes a successful catalogue entry. Verification checks the payload size, checksum and SQLite integrity. Vault verification also authenticates every secret and TOTP record with the snapshot password.

To protect a workspace snapshot with your crypt key:

```
ish backup create @notes --encrypt
```

For a vault:

```
ish backup create vault --encrypt
```

The inner vault records are already encrypted. The optional outer encryption additionally protects the complete payload file. The manifest still exposes information such as kind, timestamp and source label/path.

A successful create includes an actual decrypt-and-validate round trip for an encrypted snapshot. This takes time and temporary space. With protected keys, a workflow can prompt for the key password more than once because encryption and verification are separate steps. Passwords are not cached between commands.

Backup list reads catalogue metadata; it is not a substitute for verifying payloads. A checksum detects accidental changes relative to the manifest. Because manifests are unsigned, someone able to replace both a plaintext payload and its checksum can defeat that check.

14 | Export to iOS Files

Export turns one verified catalogue entry into one portable uncompressed bundle:

```
ish backup export BACKUP_ID \
  --to /mnt/export/vault-2026-10-16.ishbackup
```

Choose a new filename for each export. The destination must not already exist. The export command verifies a private copy before packaging the manifest and payload, and reports the output path when complete.

Export does not add encryption. If the snapshot was created without --encrypt, its database payload remains plaintext in the bundle. A vault payload still contains encrypted records, but its names and other metadata can remain visible. For a private off-device vault backup, create it with --encrypt first.

A bundle contains exactly two regular files: manifest.json and the selected payload. Do not unpack, rename its internal files, compress it again or edit it before import. It is a data-transfer format, not an executable installer.

The export destination may be an iOS-mounted filesystem that lacks the same atomic publication features as the private iSH filesystem. Export uses exclusive creation and reports success only after completing the write. An interruption can leave an incomplete external file; do not treat file existence alone as proof of completion. Import verification is the next check.

Keep your protected private-key backup separately from the bundle, and keep its password separately from both. A bundle alone cannot recover an encrypted backup on a new installation.

A recovery drill should include copying a bundle off-device and importing it into an isolated test installation with the appropriate key material. Simply viewing its filename in Files is not a restore test.

15 | Import a portable backup

On the destination installation, install and self-test the toolkit first. For encrypted bundles, restore the matching crypt private key to ~/.crypt/id.key with private permissions before import. Do not overwrite an unrelated existing key: preserve it and plan key identities explicitly.

```
ish backup import /mnt/export/vault-2026-10-16.ishbackup
ish backup list
ish backup verify BACKUP_ID
```

The importer rejects unsafe member names, symlinks, hard links, duplicate members, extra members, unsupported manifests and mismatched payload metadata. It copies into private staging instead of calling a general archive extraction routine. It then checks the checksum, decrypts if needed, checks database integrity and authenticates vault records before publishing the entry.

For a vault import, enter the password used by that snapshot. For protected outer encryption, also enter the crypt-key password when prompted. These checks happen before success is reported.

A duplicate ID is refused. This prevents silent replacement of an existing catalogue entry. Importing your own export back into the same catalogue will therefore fail if the original entry is still present; that is expected, not corruption. Test on a separate installation or isolated environment rather than deleting your only good backup to make the test pass.

Import adds a backup entry. It does not overwrite the active vault, change workspace profiles or restore files automatically. Choose a separate restore or guided recovery as the next deliberate action.

Never import the toolkit source .tar.gz with this command. Source installation and backup import are different operations with different formats.

16 | Restore to a separate destination

A normal restore creates a new destination and leaves the active source alone. This is the preferred first step when inspecting a snapshot.

For an ordinary database:

```
ish backup restore BACKUP_ID \
  --to ~/notes-recovered.db --preview
ish backup restore BACKUP_ID --to ~/notes-recovered.db
ish db tables ~/notes-recovered.db
```

Preview shows a plan. It does not authenticate the payload or prove the restore will succeed. The actual restore verifies the private copy it will use. An existing destination is refused even if it looks empty.

After inspecting the restored database, register it under a new workspace name if useful:

```
ish workspace create recovered-notes \
  --database ~/notes-recovered.db
```

For a vault, --to names a new directory:

```
ish backup restore BACKUP_ID --to ~/recovered-vault
ish vault --directory ~/recovered-vault verify
ish vault --directory ~/recovered-vault list
```

The restored vault is not activated. Use its snapshot password. This lets you inspect names and retrieve selected secrets before deciding whether a full active replacement is appropriate.

Restore never writes into the active ~/.vault tree. Interrupted restores can leave a new directory or private staging files. Inspect the result and choose a different new destination for a retry rather than assuming an existing partial folder is complete.

Keep enough free space for the encrypted payload, decrypted snapshot and destination copy. Memory-bounded encryption does not eliminate these disk copies.

17 | Guided active-vault recovery

Use guided recovery when you deliberately want the snapshot to become the active vault. It replaces contents; it does not merge new entries with old ones.

First inspect the snapshot through a separate restore if possible. Stop old vault/TOTP clients. Then view the plan:

```
ish backup recover-vault BACKUP_ID --preview
```

To execute:

```
ish backup recover-vault BACKUP_ID
```

The command asks you to type RECOVER. It then requests the snapshot password and, if an active v2 vault exists, that vault's current password. Verification is required; this is not a way to bypass a forgotten active password.

Before replacement, the current active vault is copied, verified and synced under ~/.ish-toolkit/recovery/before-... . The result reports this directory. Existing active records and metadata are then replaced together in one SQLite transaction, under the same vault lock and in the same database file.

After success, use the snapshot's password:

```
ish vault verify
ish vault list
```

The preserved old vault still uses its old password. You can inspect it with ish vault --directory PATH verify or get. Keep the reported directory until you are satisfied with the recovery.

If no active vault exists, recovery publishes a verified new vault. If legacy vault files are present, it refuses rather than silently overwriting them. A damaged current vault also needs careful diagnosis and preservation; the command will not discard it merely because authentication fails.

18 | Interruption and failure recovery

The meaning of an interrupted command depends on where it stopped. Do not infer success from a directory or file merely existing.

For a vault password change or active replacement, SQLite commits metadata and records together. Reopening with the toolkit lets SQLite recover a hot rollback journal. Test the relevant old and new/snapshot password, then run vault verify. Never manually delete the journal or edit vault_meta.

For recovery of an existing vault, a verified pre-recovery copy is created before the active transaction begins. Look under ~/.ish-toolkit/recovery for the retained before-... directory. It can be inspected separately with the old password. A process kill after commit can produce a completed replacement even if the terminal never printed success.

Key protection also uses atomic replacement. If an error occurs after replacement but during directory syncing, the new protected key may already be present. Check key-status and try the intended new password. Never generate a replacement identity to fix a password error: that new key cannot decrypt old files.

Backup creation and import only publish catalogue entries after verification. A hard kill can leave hidden private staging folders. Treat them as incomplete work until inspected. Export can leave an incomplete external bundle; re-export to a new name and verify through import in an isolated recovery environment.

A storage lock failure means no protected operation proceeded. The iSH EPERM compatibility handler does not disable locking. Do not delete .vault.lock or .key.lock to force access. Stop the competing process or diagnose the failed first acquisition.

19 | A sustainable backup routine

After meaningful changes, create a snapshot and verify it. Export important recovery points off-device. Keep several dated versions so an unnoticed mistake does not immediately replace your last good state.

A practical manual routine is:

```
ish vault verify
ish backup create vault --encrypt
ish backup list
ish backup verify BACKUP_ID
ish backup export BACKUP_ID \
  --to /mnt/export/vault-unique-date.ishbackup
```

Use an actual unique date or other new name. The example filename is not an automatic timestamp generator. The toolkit refuses to overwrite it if it already exists.

Record which vault password and crypt identity belong to each recovery set without putting the passwords into the manifest or a public note. Keep the protected private-key backup and its password available through separate secure means. Re-test recovery after changing either password or moving to another device.

This release has no automatic scheduler, retention policy or pruning command. It does not claim that an iSH process will continue indefinitely in the background. You decide when to run backups and which old files to retain.

Do not delete the last known-good snapshot merely because a newer file exists. Prefer evidence: creation succeeded, verification succeeded, export completed and an isolated recovery drill worked.

Deletion and the shred utility cannot guarantee forensic erasure on flash storage, filesystem snapshots or cloud-synced copies. Plan protection at creation time through encryption rather than relying on later overwriting.

20 | Common errors and what they mean

No such file during extraction: inspect /mnt/export. Your iOS downloads have previously gained an extra .tar suffix. Use the exact name; do not continue to install.sh after extraction fails.

Unknown command: check ish --version and command -v ish. Export ~/.local/bin into PATH, run hash -r and reinstall from the new source folder. A missing new command usually means an old launcher is still being used.

Wrong password: identify which prompt is active. A snapshot vault password can differ from today's vault password. The crypt-key password is separate. A matching public key cannot replace the private key.

Destination exists: choose a new restore/export name. These commands intentionally refuse overwrite. Duplicate backup ID on import means that catalogue identity is already present.

Checksum mismatch or authentication failure: preserve the suspect input and try a known-good copy. Do not edit a manifest to make its checksum match an unexplained change. That removes evidence rather than repairing the data.

Permission denied or lock failure: use the included self-test. The observed iSH competing-lock EPERM case is handled, but a failed initial lock acquisition is still a real failure. Files in private state must belong to the current user and must not be symlinks or unexpected hard links.

Disk full: free space or use a smaller dataset. Snapshot, encryption, validation and restore can require several simultaneous copies. The 60 GiB format cap is not a promise that your phone can process a file that size.

When reporting a fault, share the command name, version and error text. Remove secret values, private keys, passwords and TOTP seeds from the report.

21 | Command reference

The following are the shipped command families. Arguments in capitals are placeholders; paths with spaces must be quoted.

```
ish --version
ish selftest [--full]
ish doctor
ish db doctor

ish vault init|migrate|list|check|verify|passwd
ish vault set|get|delete NAME
ish vault --directory PATH verify
ish totp add NAME
ish totp list
ish totp [NAME]
ish totp delete NAME

ish workspace create NAME [--database PATH]
ish workspace list
ish workspace status|path|open NAME

ish db init PATH
ish db tables PATH_OR_@NAME
ish db schema PATH_OR_@NAME TABLE
ish db query PATH_OR_@NAME --sql SQL
ish db backup SOURCE NEW_DESTINATION
ish db restore SOURCE NEW_DESTINATION

ish backup create SOURCE [--encrypt]
ish backup list
ish backup verify ID
ish backup restore ID --to PATH [--preview]
ish backup export ID --to FILE.ishbackup
ish backup import FILE.ishbackup
ish backup recover-vault ID [--preview]

ish crypt keygen|key-status|protect-key|pubkey
ish crypt encrypt INPUT [NEW_OUTPUT]
ish crypt encrypt -r PUBLIC_KEY_FILE INPUT
ish crypt decrypt INPUT [NEW_OUTPUT]
ish crypt encrypt-dir DIRECTORY
```

Database writes require --write and --allow-destructive together. Query also supports --file and --params-file. SOURCE for backup create is vault, a database path or @workspace. Use subcommand --help where available for parser details.

22 | Format notes and trust model

The active v2 vault is SQLite with encrypted record values plus salt, verifier and fixed KDF metadata. Its master-key derivation retains Argon2id parameters from v2: time cost 3, memory 65536 KiB, parallelism 1 and 32-byte output. Record encryption uses AES-GCM. Names are authenticated with their record values but remain visible metadata.

CRYPTv2 files start with an eight-byte magic value, then a 16-byte salt, 32-byte ephemeral X25519 public key and 12-byte nonce. X25519 shared material is expanded with HKDF-SHA256 using the crypt-v2 context. AES-256-GCM authenticates the header and ciphertext; the final 16 bytes are the tag. New ephemeral material is generated for each encryption. This is an application container, not a claim of independent protocol certification.

Decryption uses bounded chunks and authenticates the complete stream before publishing the requested output. Private temporary plaintext can still exist during processing. The compatible CRYPTv1 reader uses its original ChaCha20-Poly1305 construction and bounded whole-file processing.

Protected private keys use cryptography's encrypted PKCS8 serialization. Its library-selected encryption parameters can vary with the installed backend. The key identity remains X25519; the wrapping password is not a new recipient key.

Official implementation references used for the crypto interfaces:

- https://cryptography.io/en/stable/hazmat/primitives/symmetric-encryption/

- https://cryptography.io/en/stable/hazmat/primitives/asymmetric/x25519/

- https://cryptography.io/en/stable/hazmat/primitives/asymmetric/serialization/

This release protects against accidental overwrite, many corruption cases and unverified restoration. It does not protect an already compromised account, provide tamper-proof manifests, recover lost keys/passwords, or guarantee forensic deletion. Those boundaries are part of using the build correctly.